

CSCI 3150 - Introduction to Operating Systems (Reading Notes)

• Operating Systems

- A system software that manages computer hardware and software resources, and provides common services for computer programs.
- Abstract access to hardware resources, e.g. CPU, memory, communications, I/O devices
- Manage resources (Allocation, Protection, Reclamation, Virtualization)

• Unix Structure

- Applications (written by programmer, compiled by programmer, uses library calls)
- Libraries (written by elves, uses system calls)
- Portable OS Layer (system calls, read/open/..., high-level)
- Machine-dependent Layer (Bootstrap, Initialization, Interrupt & Exception, I/O device driver, Memory management, Kernel-User mode switching, Processor management)

• User / Kernel-mode [User-mode: User programs; Kernel mode: operating system, protected instructions, set mode status bit]

- Supported by hardware (stability, security, prevent unauthorized access to critical resources)
Efficient privilege level transitions Completely on hardware (Base-metal execution)
- Software-only: significant performance overhead due to frequent checks and context switches.

```
if ([PC] != protected_instruction)
    execute(PC);
else
    switch_to_kernel_mode();
```

Processor privilege levels stored in control registers restrict access to privileged instructions. When an **exception** is generated, the exception transfers control to OS.

• Hardware Features for OS

User / Kernel-mode protections, memory protection, system calls, interrupt and exceptions, timer, I/O control, synchronisation

Halting (system-wide disruption and instability) control registers

prevent conflicts and security risks, fair access

• Memory Protection Mechanisms

- Base and limit registers
- Page table pointers, page protection, TLB
- Virtual memory
- Segmentation

• Events - triggers the entry to kernel

- ① Stop current execution
- ② Switch to kernel mode, call event handler

Unnatural change to control mode

Specific types are defined by machine.

→ transfer control to handler within OS

• saves program state (PC, registers, etc.)

• handler functionality invoked

• restores program state, return to program.

• Two general categories of events

- Interrupt: signals sent to the processor by hardware devices or external resources to request immediate attention.

Example: completion of I/O operations, handling hardware errors, context-switch in multitasking

Asynchronous: occur outside the normal flow of program
External HW or OS

- Exception: events that occur during execution of a program that disrupt the normal flow of instructions

Example: division by zero, invalid memory access. Typically **synchronous** events caused by the **program**

• Reasons for triggering events

Unexpected events: unexpected failure

Unexpected

Fault

Deliberate

Syscall trap

Deliberate events: scheduled by OS / application

Interrupt

Software interrupt

• Faults in application

Hardware detects and reports "exceptional" conditions

"faults": save state (PC, registers, mode) handled by "fixing" or notifying

after handling, process can be restarted. otherwise halt process, write state to file and destroy

• Faults in kernel

Fatal crash in system - halted core dumped locked up machine

• System Calls in Kernel: Requirements

① State is saved - resume to interrupted process

② verify arguments - avoid buffer overflow, invalid memory access

③ table of system calls - **efficient** dispatching, **security**, extensibility

• Finding Interrupt Handlers

IDT (interrupt descriptor table) set up at boot time

handler = IDT[intr-number].

• Timer

- fallback mechanism for OS to reclaim control

- timer expires $\rightarrow$ interrupt

- basis for kernel event scheduler

- prevents infinite loops, sleep() function

• I/O control

- initiating/completing
memory mapping

context-switch to kernel (CPU looks up kernel routine address / offset / vector)

• Interrupt

- The OS can be interrupted to handle critical tasks efficiently

- Alternative approach: actively checks status the status of devices at regular interval

Drawbacks: wastes CPU resources, latency, complexity (large number of devices)

• Process: a program in **execution** separated from other instances

Execution state ⁽¹⁾ **Running**: Executing instructions on the CPU

⁽²⁾ **Ready**: Waiting to be assigned by the CPU

⁽³⁾ **Waiting**: Waiting for an event, e.g. I/O completion

• Process

(i) Most of the time in **waiting state**

(ii) The number of process in running state is **constrained by the number of cores on the CPU**.

(iii) **Concurrency enables better resource utilization on multi-core CPU**.

Representation in Kernel (**PCB - process control block**)

Pointer	Process state
	process id
	program counter
	registers
	memory limits
	accounting
	file status

→ time limit / process number / owner

• Context Switching: Changing CPU hardware state from one process to another

Suspend: saves current value of registers to PCB

ready: loads back hardware registers from PCB

• State Queues: Each PCB is queued on a state queue according to its current state
Dynamically allocated in OS memory, deallocated when terminated

• Process Creation

Created by a parent process

Init pid 1 is the first (user space) process

New **PCB**, **address space** (inherited from parent process) **kernel resources pointed to parent's resources**
Ready queue

fork() returns twice $\left\{ \begin{array}{l} \text{PID of child to parent} \\ 0 \text{ to children} \end{array} \right.$

by **eax** on 32-bit architectures

or **PCB** stores the states of all registers

• Terminating a process ⁽ⁱ⁾ close, terminate threads ⁽ⁱⁱ⁾ open files ⁽ⁱⁱⁱ⁾ allocated memory ^(iv) remove PCB

A process does not need to clean up itself - OS handles it: reclaiming resources for resource management

Only a parent can remove the PCB and "reap" - call wait(), check for the process's exit status

wait() (wait for a single object) waitpid() suspends till the specified child process ends

• Processes to Threads

A process includes ⁽ⁱ⁾ address space ⁽ⁱⁱ⁾ OS resources, accounting information ⁽ⁱⁱⁱ⁾ execution state (PC, SP, registers)

Costly: space allocation, initialization cost, communication between processes (overheads of system calls, data copying)

• Parallel programs

- Efficient and cooperative

Same code, data (address space), privileges, resources

Each has its own execution state: PC, SP, registers (Execution states) ^{thread}

- A thread is bound to a single process, the unit of scheduling

• Thread Model

- Shared information: processor information, memory, I/O and file

- Kernel level threads: implemented in kernel, managed by OS ^{informed scheduling}

- User level threads: managed by run-time system (user-level library) ^{uninformed scheduling}

Small, cheap and fast (PC, register, stack, small TCB)

Creation, switch, synchronization all by procedure calls. no kernel involvement

- Issues with Threads

① Interface

thread-create (procedure, arg) new thread

thread-yield () voluntarily give up the processor

thread-exit () terminate the calling thread

thread-join (target-thread) suspend the execution of calling thread until target-thread terminates

② Scheduling

user-level thread: entirely in user space

determines when a thread runs by a queue

Run queue: Threads currently running

Ready queue: Threads ready to run

"Wait queue": condition variables, blocking queues, semaphore, joining

③ Pre-emptive / Non-preemptive scheduling

Pre-emptive scheduling: temporary interrupt an executing task by an external scheduler

Running → Ready with the intention of resuming it at a later time

Waiting → Ready fully determined by OS to ensure every process uses the same amount of CPU time

Non-preemptive scheduling: any new process has to wait until the running process finishes its CPU cycle

Termination voluntarily yield the CPU

Running → Waiting

• Context Switch in Thread

- Push all machine state onto its stack

- Restore content for next thread (pop all machine state from its stack)

- Return to caller

• Synchronization

- Threads access the shared data, thus need process sequencing
- Producer-Consumer problem

Producer: produce 1 cookie per person

◦ Step 1: increment the counter on the board

◦ Step 2: put one cookie on the table

critical region

switch to consumer

Consumer:

◦ Step 1: read the counter

◦ Step 2a: if the counter is zero, go back to step 1

◦ Step 2b: if the counter is non-zero, take a cookie from the table

switch to producer

◦ Step 3: decrement counter on the board

Rule: only one should "operate" at any time

Race condition due to concurrency and data sharing (cookie, counter)

Depends on one point reaching x before another reaching y .

• Shared Resources

- Local variables (private) on the stack
- Dynamic objects (public) shared on the heap (as long as pointers are not shared)
- Global variables (public) on the data segment

• Interleaved Schedule

```
withdraw(account, amount) {  
    balance = get_balance(account);  
    balance = balance - amount;  
    put_balance(account, balance);  
    return amount;  
}
```

```
withdraw(account, amount) {  
    balance = get_balance(account);  
    balance = balance - amount;  
    put_balance(account, balance);  
    return amount;  
}
```

Execution Sequence
↓

```
balance = get_balance(account);  
balance = balance - amount;
```

```
balance = get_balance(account);  
balance = balance - amount;  
put_balance(account, balance);
```

```
put_balance(account, balance);
```

context switch

(mutex)

Motivation: mutual exclusion for synchronization $\Rightarrow$ larger atomic blocks (critical region)

• Critical Region Requirements

Process/Thread {

while (True) {

// Enter Critical Region

Access shared variables // Critical section

// Leave Critical Region

Do other work

}

}

• mutual exclusion

• progression

(i) A thread in the critical region will have to leave

(ii) A thread not in the critical region cannot prevent the other from entering

• bounded waiting time

• no assumption on speed or CPU

• Critical Region Mechanisms

- Atomic R&W

- Locks

- Semaphores

- Monitors

- Message

• Locks

- Interface

acquire(): before entering critical region

release(): after entering critical region

① Between acquire() and release(), the thread holds the lock

② acquire() does not return until any previous holder releases (blocked)

③ acquire() and release() must be paired to avoid deadlocks

- Spinlock Implementation

struct lock {

int held = 0;

}

void acquire(lock) {

while (lock->held);

lock->held = 1;

}

void release(lock) {

lock->held = 0;

}

busy-wait (spin-wait) for lock to be released

Change: while (test-and-set (&lock->held));

Atomic

wasteful

Sol. 1) thread_yield()

2) sleep and wake up (locking)

critical region!
(race condition)

Requires HW support! (test-and-set = disable/enable interrupts)

- Disabling interrupts = issues
 - (i) Multithread disabling/enabling conflict
 - (ii) Disabling global interrupts misses important information
 - (iii) Multiprocessor

- Peterson's Algorithm without hardware

int turn = 1; bool try1 = false, try2 = false;

```
while (true) {
    try1 = true;
    turn = 2;
    while (try2 && turn != 1);
    critical section
    try1 = false;
    outside of critical section
}
```

```
while (true) {
    try2 = true;
    turn = 1;
    while (try1 && turn != 2);
    critical section
    try2 = false;
    outside of critical section
}
```

Condition Variables

- Checks whether a condition is true before continuing its execution
- join() - a parent thread wishes to wait for the completion of a child
- Implementation 1 - Spin-based approach

```
bool done = 0;
void * child(void * arg) {
    printf("child\n");
    done = 1;
    return NULL;
}

int main() {
    printf("parent begins\n");
    pthread_t c;
    pthread_create(&c, NULL, child, NULL);
    while (done == 0); // spin
    printf("parent: end\n");
    return 0;
}
```

- Implementation 2 - Condition Variable

- Queue of threads

- **Waiting**: an explicit queue that threads can put themselves on when some state of execution is not as desired.

- **Signaling**: other thread **changes its state** and wakes up **at least one** of the waiting threads and allow them to continue.

Order: unspecified.

(i) Condition variable

(ii) State variable

(iii) lock for protecting the shared state variable

- API routines

`pthread_cond_t c;`

`pthread_cond_wait(pthread_cond_t *c, pthread_mutex_t *m);`
`pthread_cond_signal(pthread_cond_t *c);`

`wait()` releases lock and put calling thread to sleep.

the waken thread reacquire lock before returning to caller

```
int done=0;
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t c = PTHREAD_COND_INITIALIZER;
void thr_exit() {
    pthread_mutex_lock(&m);
    done=1;
    pthread_cond_signal(&c);
    pthread_mutex_unlock(&m);
}
```

```
void *child(void *arg) {
    printf("child\n");
    thr_exit();
    return NULL;
}
```

```
void thr_join() {
    pthread_mutex_lock(&m);
    while (done == 0)
        pthread_cond_wait(&c, &m);
    pthread_mutex_unlock(&m);
}
```

```
int main(int argc, char *argv[]) {
    printf("parent: begin\n");
    pthread_t p;
    pthread_create(&p, NULL, child, NULL);
    thr_join();
    printf("parent: end\n");
    return 0;
}
```

Poor implementations:

<pre>void thr_exit() { pthread_mutex_lock(&m); pthread_cond_signal(&c); pthread_mutex_unlock(&m); }</pre>	<pre>void thr_join() { pthread_mutex_lock(&m); pthread_cond_wait(&c, &m); pthread_mutex_unlock(&m); }</pre>	A
---	---	---

Without variable done, if child runs first and parent thread waits, no thread wakes parent up!

<pre>void thr_exit() { done=1; pthread_cond_signal(&c); }</pre>	<pre>void thr_join() { if (done == 0) pthread_cond_wait(&c); }</pre>	B
---	--	---

Race condition: Parent calls join to wait, and got interrupted.

- Child signals

- Parent sleeps (can't be woken up)

• Producer/Consumer: Single CV and While

[Handwritten for better understanding]

```
pthread_cond_t c = PTHREAD_COND_INITIALIZER;
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
void *producer(void *arg) {
    int i;
    for (i=0; i<loops; i++) {
        pthread_mutex_lock(&m);
        while(count==1)
            pthread_cond_wait(&c, &m);
        put(i);
        pthread_signal(&c);
        pthread_mutex_unlock(&m);
    }
}
```

```
void *consumer(void *arg) {
    int i;
    for (i=0; i<loops; i++) {
        pthread_mutex_lock(&m);
        if (count==0)
            pthread_cond_wait(&c, &m);
        int j = get();
        pthread_cond_signal(&c);
        pthread_mutex_unlock(&m);
    }
}
```

```
int buffer, count=0;
void put(int val) {
    assert(count==0);
    count = 1;
    buffer = val;
}
int get() {
    assert(count==1);
    count = 0;
    return buffer;
}
```

More than one consumer/producer?

- Two consumer, One producer no guarantees about the order
C₁ gets ready but C₂ consumes the buffer!

• Solution: Many Producers Many Consumers

producer threads wait on the condition empty and signals fill
consumer threads wait on fill and signal empty

```
cond_t empty, fill;
mutex_t mutex;
void *producer(void *arg) {
    int i;
    for (i=0; i<loops; i++) {
        pthread_mutex_lock(&mutex);
        while(count==1) pthread_cond_wait(&empty, &mutex);
        put(i);
        pthread_cond_signal(&fill);
        pthread_mutex_unlock(&mutex);
    }
}
```

```
void *consumer(void *arg) {
    int i;
    for (i=0; i<loops; i++) {
        pthread_mutex_lock(&mutex);
        while(count==0) pthread_cond_wait(&fill, &mutex);
        int tmp = get();
        pthread_cond_signal(&empty);
        pthread_mutex_unlock(&mutex);
        printf("%d\n", tmp);
    }
}
```


• Semaphore

- Mutex & Atomic counters
- P(semaphore), wait()
- V(semaphore), signal(), post()

sem_getvalue(sem_t *sem, int *value);

```
#include <semaphore.h>
```

```
sem_t s;
```

```
sem_init(&s, 0, 1); // initialize s to value 1.
```

```
// semaphore is shared among all threads in the same process
```

```
sem_wait(&m);
```

```
// critical section
```

```
sem_post(&m);
```

```
int sem_post(sem_t *s) {
```

increment the value of semaphore s by one

if there are one or more threads waiting, wake one

```
}
```

```
int sem_wait(sem_t *s) { if the value of semaphore is ≥ 1,  
return right away
```

decrement the value of semaphore s by one

wait if value of semaphore s is negative

```
}
```

the value of semaphore is equal to the number of waiting threads.

- Bounded Buffer Problem with Semaphore

```
int buffer[MAX]; int fill=0, use=0;
```

```
void put(int value) {
```

```
    buffer[fill]=value;
```

```
    fill = (fill+1) % MAX;
```

```
}
```

```
int get() {
```

```
    int tmp=buffer[use];
```

```
    use = (use+1) % MAX;
```

```
    return tmp;
```

```
}
```

```
int main(int argc, char *argv[]) {
```

```
    sem_init(&empty, 0, MAX);
```

```
    sem_init(&full, 0, 0);
```

```
    pthread_t c, p;
```

```
    pthread_create(&c, NULL, consumer, NULL);
```

```
    pthread_create(&p, NULL, producer, NULL);
```

```
    pthread_join(&c, NULL);
```

```
    pthread_join(&p, NULL);
```

```
    return 0;
```

```
sem_t empty, full;
```

```
void *producer(void *arg) {
```

```
    int i;
```

```
    for (i=0; i<loop; i++) {
```

```
        sem_wait(&empty);
```

```
        put(i);
```

```
        sem_post(&full);
```

```
    }
```

```
}
```

```
void *consumer(void *arg) {
```

```
    int i, tmp=0;
```

```
    for (i=0; i<loop; i++) {
```

```
        sem_wait(&full);
```

```
        tmp=get();
```

```
        sem_post(&empty);
```

```
    }
```

```
}
```

~~mutex semaphore wait~~

~~mutex semaphore post~~

not atomic!

~~mutex semaphore~~

~~mutex semaphore~~

• Reader-Writer Locks

```

typedef struct _rwlock_t {
    sem_t lock; // binary semaphore
    sem_t writelock; // allow writer or readers
    int readers;
} rwlock_t;

void rwlock_init(rwlock_t *rw) {
    rw->readers = 0;
    sem_init(&rw->lock, 0, 1);
    sem_init(&rw->writelock, 0, 1);
}

void rwlock_acquire_readlock(rwlock_t *rw) {
    sem_wait(&rw->lock);
    rw->readers++;
    if (rw->readers >= 1)
        sem_wait(&rw->writelock);
    sem_post(&rw->lock);
}

void rwlock_release_readlock(rwlock_t *rw) {
    sem_wait(&rw->lock);
    rw->readers--;
    if (rw->readers == 0)
        sem_post(&rw->writelock);
    sem_post(&rw->lock);
}

void rwlock_acquire_writelock(rwlock_t *rw) {
    sem_wait(&rw->writelock);
}

void rwlock_release_writelock(rwlock_t *rw) {
    sem_post(&rw->writelock);
}
    
```

critical
region

critical
region

- Only a single writer can acquire the lock
- Once a reader has acquired a read lock, more readers will be allowed to acquire the read lock too
- A writer will have to wait until all readers have finished

• Concurrency Bugs

- Non-deadlock

(i) Atomicity sol. Add locks

(ii) Order sol. CV

- Deadlock

cyclic dependence

reason: (i) complex dependencies between components

(ii) encapsulation

- Conditions for Deadlock

- | | | |
|---|------------------|---|
| 1 | Mutual Exclusion | Threads claim control of wanted resources |
| 2 | Hold-and-Wait | Thread holds resources while waiting for additional resources |
| 3 | No Preemption | Resources cannot be forcibly removed from threads that are holding them |
| 4 | Circular Wait | Circular chain of request |

sol. 4) always execute p1 before p2.

2) acquire lock all at once

- require us to know when calling a routine exactly which locks must be held and to acquire ahead of time
- decreases concurrency

```
lock(prevention);  
lock(L1);  
lock(L2);  
...  
unlock(prevention);
```

3) trylock()

grab the lock if available

returns -1 for later attempts

b) livelock

add a random delay

1) lock-free approach by HW.

misc. Scheduling (requires global knowledge)

Detect and recover Allow occasional deadlocks (reboot OS if frozen)

Interval checks: build resource graph and check cycle

• CPU Scheduling Assumptions

1. Each CPU runs for the same amount of time
2. All jobs arrive at the same time
3. All jobs only use the CPU (performing no I/O operations)
4. The run-time of each job is known

• Scheduling Metrics

- Job completion time = $JCT = T_{done} - T_{arrival}$
- Fairness
- Response time = $T_{first\ run} - T_{arrival}$ (for SRTF)

• Greedy Strategies

- First-In First-Out (Counter: Each job no longer runs for the same time)
- Shortest Job First (SJF) (Counter: jobs do not have to arrive at the same time)
Non-preemptive scheduler
- Shortest Remaining Time First (SRTF)
Preemptive (Interrupts a task)
New job enters system: (i) Determine the remaining time of all jobs
(ii) Schedule the job with least time left

- Round-Robin (RR)

Fair: run a job for a timeslice and switch to next job in the run queue till all jobs are finished

Length of timeslice must be a multiple of the timer interrupt period.
Short response, poor JCT

↕
Short timeslice

• Multi-Level Feedback Queue (MLFQ)

- Motivation: No prior knowledge about the length of a job

- Key Idea

Each job is assumed to be short, latency-insensitive job
(Highest priority)

Keeps running — decrease priority level

Waiting for I/O — keep its priority level

$MLFQ \approx SJF$

Rule 1: If $Priority(A) > Priority(B)$, run job A

Rule 2: If $Priority(A) = Priority(B)$, run job A and B in RR

Rule 3: When a job enters a system, place it at highest priority

Rule 4: Running the entire timeslice, demoted;

Voluntarily give up the CPU, keep it on same priority

Rule 5: Move all jobs back to top priority after a time period of S.

- Issues:

Starvation: too many interactive jobs in the system
CPU time will not be given to long-running jobs

Cheater: Issue an I/O after 99% of CPU time

Sol. Lower priority, longer time slice

Program may change behavior

Sol. Rule 5

• Lottery sharing / Proportional Share Scheduler

- Fair share scheduler, each job obtains a certain percentage of CPU time

- Tickets represents a scheduling opportunity:

Percentage of tickets represents a process's share of the system resource

- Scheduler picks a winning ticket, loads the process and execute (probabilistic)

- Mechanism

Ticket currency: distribute among a process's own jobs, the system does auto conversion

Ticket transfer: a process can temporary lend tickets to another process

Ticket inflation: boost tickets if require more CPU time

- List implementation: keep processes in a list and scan

Fairness metric: All jobs finishes at nearly the same time

• Deterministic Approach: Stride Scheduling

Stride of each process = $\frac{\text{a constant large number}}{\text{the number of tickets}}$

Pick the process with the smallest stride to run

```
Set the pass of all clients to 0
current = remove_min(queue);
schedule(current);
current->pass += current-stride;
insert(queue, current);
```

- Job with 0 pass entering the system will monopolize the CPU

- Requires to store the state of every job

• The Linux Completely Fair Scheduling (CFS)

- Nice Value and Weight Function

Nice value $\downarrow \Rightarrow$ priority $\uparrow$ (weight $\uparrow$) $\approx \frac{1024}{1.25^{\text{nice}}}$

- Non-Fixed Timeslice

$\text{timeslice}_i = \frac{\text{weight}_i}{\sum_i \text{weight}_i} \cdot \text{sched_latency}$ (usually 48 ms)

minimum granularity: at least 6ms per slice

- vruntime

Ensure that the vruntime of every process are balanced

Smaller vruntime $\Rightarrow$ higher CPU demand $\text{vruntime_delta} = \frac{\text{weight}_0}{\text{weight}_i} \cdot \text{runtime}_i$

- Red-Black Tree $O(\log n)$
find processes with minimum virtual runtime

Ensure that any processes that sleeps has vruntime no smaller than min-runtime in the running queue

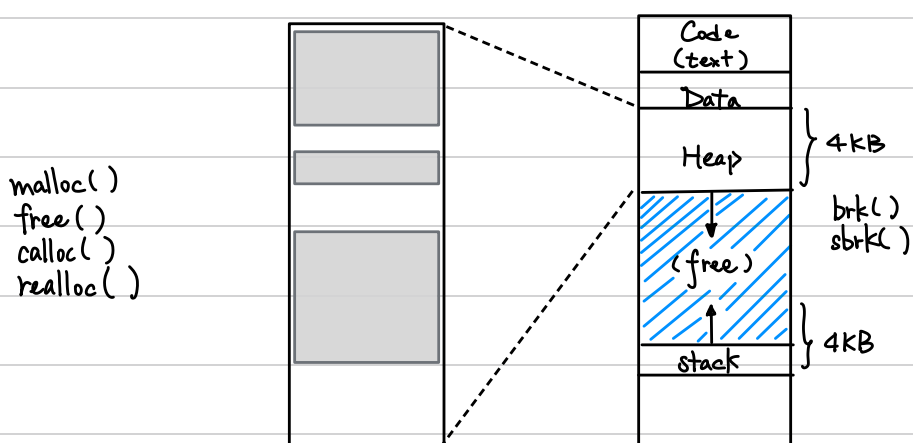
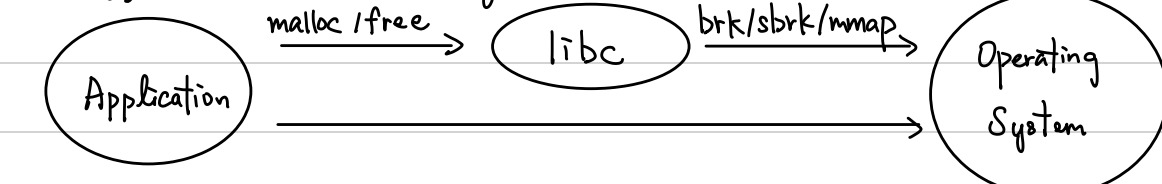
• Dynamic Memory

- malloc() / free()
- calloc() / realloc()

> in libc to manage memory in heap

- brk() / sbrk() ——— system call . to change the heap size

brk() and sbrk() are used to adjust the "break" point (end address of the heap)



```
#include <unistd.h>
int brk(void *addr)
void *sbrk(int ptr_t increment)
```

Ask OS to expand the program's break if there lacks heap space

Memory Allocation

```
#include <stdlib.h>
void *malloc(size_t size)
```

size_t : unsigned int, size of memory block in bytes

returns SUCCESS or FAIL
(void pointer) (null pointer)

Memory Block Size

```
sizeof()
```

actual size of variables :

size known at run-time $\text{int } *x = \text{malloc}(10 * \text{sizeof}(\text{int}));$

size known at compile-time $\text{int } x[10]; \text{ printf}("%d \ln", \text{sizeof}(x));$

free a memory region allocated by a call to malloc

ptr: the memory block allocated with malloc

Memory Deallocation

```
#include <stdlib.h>
void *free(void *ptr)
```

• Memory Virtualization

- Through limited direct execution (LDE)
- Efficiency and control gained by hardware support
- Base- and -Bounds registers

OS decides where to load the process in physical memory

set base register a value

physical address = virtual address + base

virtual address must not exceed the bounds or negative

$0 \leq \text{physical address} < \text{bounds register}$

OR

$0 \leq \text{virtual address} < \text{bounds register}$

a layer of protection
that prevents illegal memory
access

- Hardware Requirements

- ① Privileged mode: prevent user-mode processes from executing privileged operations
- ② Base/bounds register: Need a pair of registers per CPU to support address translation and bounds checks.
- ③ Circuitry to do translations, check within bounds limits
- ④ Privileged instructions for updating base/bounds:
OS must set these values before letting a user program run
- ⑤ Privileged instructions to register: Exception handlers to handle errors

- Critical Junctures

- ① Starts running
Allocate space in physical memory
Maintain a free list with addr & ranges of physical memory that are not in use
free list → 16KB → 48KB → 16KB → 32KB → 48KB → 64KB → 48KB
- ② Terminated process
Reclaiming memory for use
Put back the memory on the free list
free list → 16KB → 32KB → 48KB
- ③ Context switch
Save and store base-and-bounds pair
PCB stores base & bounds

- Issues of base-and-bounds

Internal fragmentation: pre-allocated huge space that contains significant unused memory
large address spaces is hard to fit into main memory

- Segmentation

A contiguous portion of the address space of a particular length

(code, stack, heap)

Can be placed in different parts of main memory: base-and-bounds pairs for each segment

Physical address = offset + base

Hardware detects out-of-bounds memory access OS issues SIGSEGV

Explicit approach to chop up address space (top few bits)

- Extra Hardware Support

- ① Stack: grow backwards — add a growth bit (0: negative; 1: positive direction)
- ② Sharing of code: protection bits (rwx)

- Issues of segmentation

External fragmentation: multiple deallocated memory blocks that are wasted

Solved through compaction: rearrange the existing segments (a costly approach)

- ① Stop current process
- ② Copy data to elsewhere
- ③ Change segment register value

• Paging

An address space is divided into fixed-sized units

Physical memory is also divided into pages called page frame

- Advantages

Flexibility: abstraction of address space

no assumption on heap and stack growth

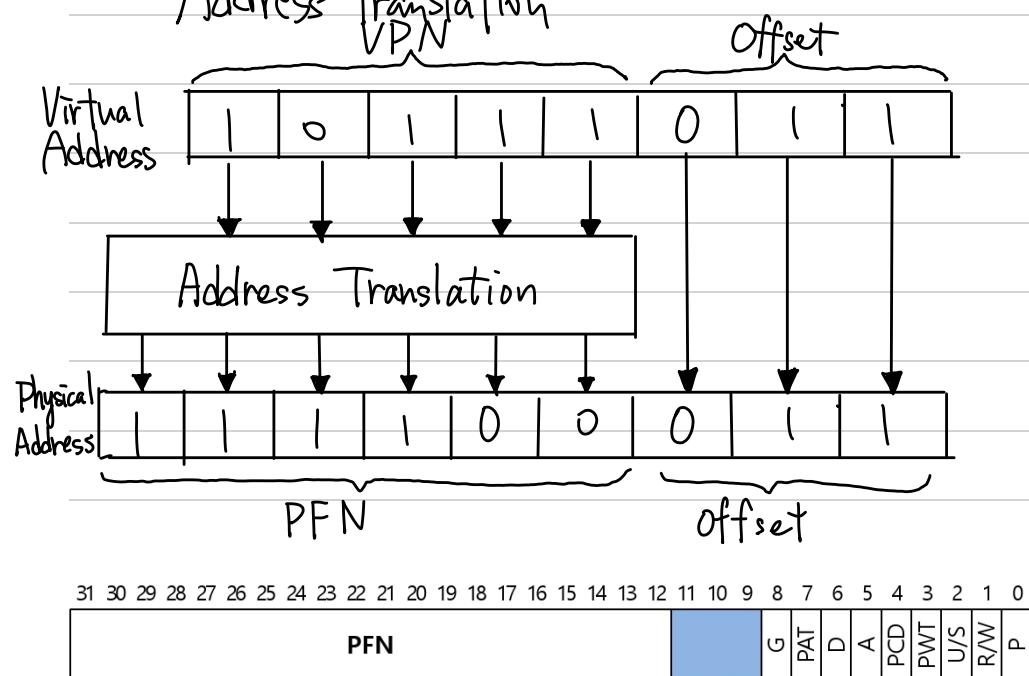
Simplicity: page and page frame are of the same size

Free list is easy to maintain; Easy to allocate memory

- Page Table: reference entries to translate virtual address to physical address

Page table is per process

- Address Translation



VPN: virtual page number

Offset: index within the page block

Page table are stored in memory, but it is sometimes too large

Indexes the array by VPN, and looks up the page-table entry (PTE)

Valid bit: indicating whether the particular translation is valid (unused space)

Protection bit: indicating whether the page could be read from, written to, or executed from

Present bit: indicating whether the page is in physical memory or on disk (swapped out)

- Inefficiency of Paging

The starting location of the page table is needed

Every memory inference has to pass through an extra layer of memory inference through the page table

- Access Memory with Paging

```
VPN = (VirtualAddress & VPN_MASK) >> SHIFT;
```

```
PTEAddr = PTBR + (VPN * sizeof(PTE));
```

```
PTE = AccessMemory(PTEAddr);
```

```
if (PTE.Valid == False)
```

```
    RaiseException(SEGMENTATION_FAULT);
```

```
else if (CanAccess(PTE.ProtectBits) == False)
```

```
    RaiseException(PROTECTION_FAULT);
```

```
else
```

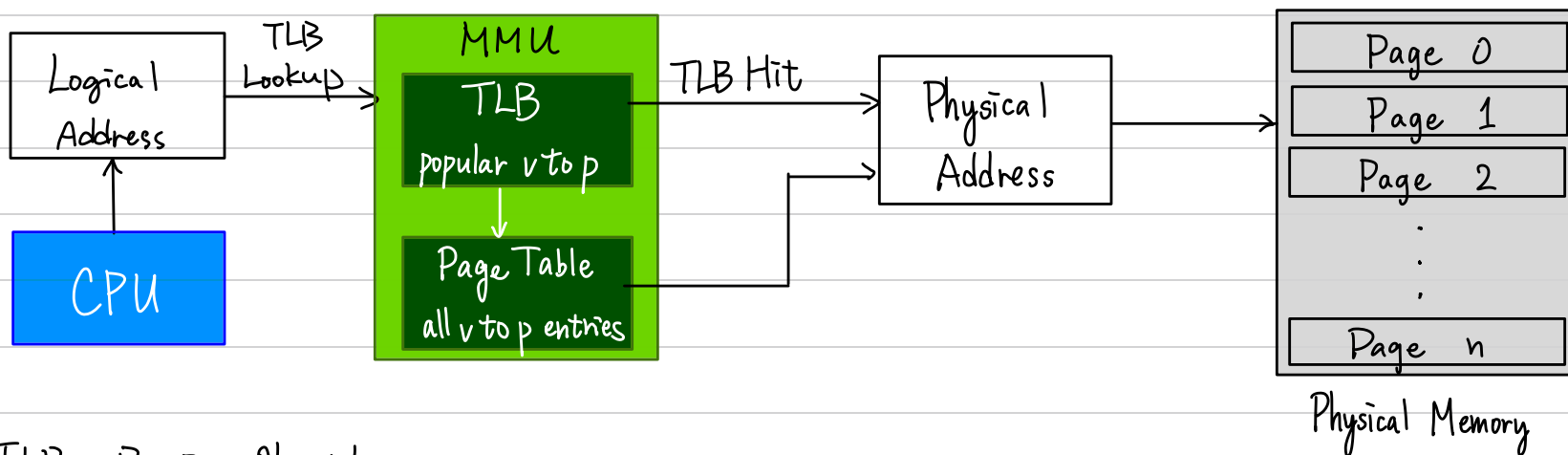
```
    offset = VirtualAddress & OFFSET_MASK;
```

```
    PhysAddr = (PTE.PFN << PFN_SHIFT) | offset
```

```
    Register = AccessMemory(PhysAddr)
```

• Translation Lookaside Buffer (TLB)

- A hardware cache in the chip's memory-management unit (MMU) for popular virtual-to-physical address translation.



- TLB Basic Algorithm

```

VPN = (VirtualAddress & VPN_MASK) >> SHIFT;
(Success, TLBEntry) = TLB_Lookup(VPN);
if (Success == TRUE) { // TLB Hit
    if (CanAccess(TLBEntry.ProtectBit) == True) {
        offset = VirtualAddress & OFFSET_MASK;
        PhysAddr = (TLBEntry.PFN << SHIFT) | offset;
        AccessMemory(PhysAddr);
    } else RaiseException(PROTECTION_ERROR);
} else { // TLB Miss
    PTEAddr = PTBR + (VPN * sizeof(PTE));
    PTE = AccessMemory(PTEAddr);
    if (PTE.Valid == False)
        RaiseException(SEGMENTATION_FAULT);
    else {
        TLB_Insert(VPN, PTE.PFN, PTE.ProtectBits);
        RetryInstruction();
    }
}
  
```

- Locality

Temporal locality: An instruction or data that has been recently accessed will likely be re-accessed again.

Spatial locality: Memory near an instruction or a data item will likely be accessed.

- TLB Miss Handler

CISC architecture: hardware walks on the page table, find correct entry, extract the translation, retry instruction.

RISC architecture: software-managed TLB, raise exception and relayed by a trap handler.

- TLB Entry

fully-associative

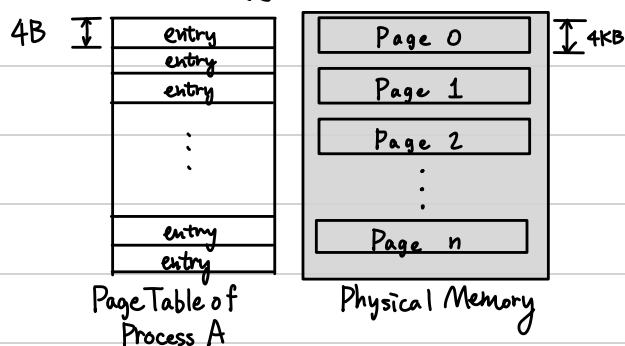
hardware searches the entire TLB *in parallel* to find the desired translation

Solving context-switch ambiguity: add an *address space identifier* (ASID) field in the TLB. (only required when active)

Reduce the number of physical pages/frames by *sharing of pages*

• Advanced Page Table

- Linear Tables



32-bit address space

$$\frac{2^{32}}{2^{12}} \times 4 = 4 \text{ MB per page table}$$

Big pages lead to *internal fragmentation* (memory allocated to the process is larger than demanded)

Hybrid approach: paging and *segmentation* - page table for each segment with base and bounds register

TLB Miss on Hybrid Approach

① The hardware gets physical address from page table

segment bits determines the base-and-bounds pair

hardware takes physical address and combines with VPN to form address of PTE

```
SN = (VirtualAddress & SEG_MASK) >> SN_SHIFT;
VPN = (VirtualAddress & VPN_MASK) >> VPN_SHIFT;
AddressOfPTE = Base[SN] + (VPN * sizeof(PTE));
```

External fragmentation: large but sparsely used heap
(non-contiguous data blocks creating a lot of gaps)

② Access main memory with PTE and offset

- Multi-level Page Tables

Page Directory

The page directory contains one entry per page of the page table

Consists of a number of entries, i.e. page directory entries (PDE)

A PDE has a valid bit and a page frame number (PFN)

Advantage

Only allocates page-table space in proportion to the amount of address space you are using

OS can grab the next free page when it needs to allocate or grow a page table

Disadvantage

Time-space trade-off

Complexity due to indirection

- Example: Single-level Paging

Consider a 16-bit virtual address with 64-byte pages, and the page table is 4B per entry

Number of page table entries = $\frac{2^{14}}{2^6} = 2^8$ (entries) $\Rightarrow$ VPN: 8 bits

Offset: $64 = 2^6$ bytes $\Rightarrow$ 6 bits

Page table size = $2^8 \times 4 = 1$ Kbyte

Physical memory requires $\frac{1KB}{64} = 16$ pages

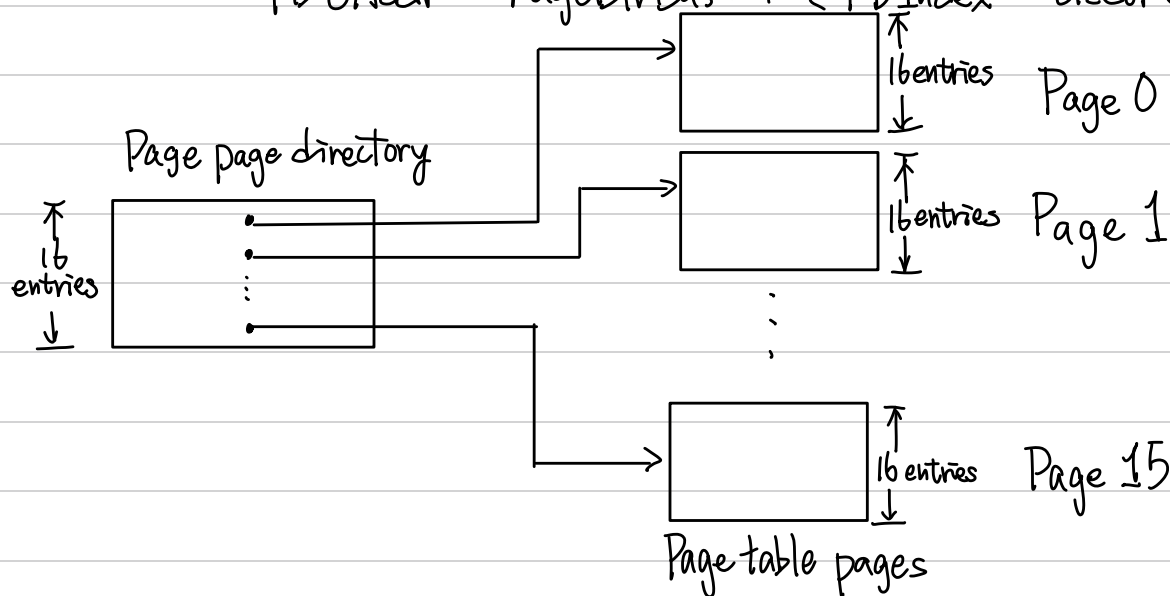
- Example: Two-level Paging

Page directory index: Create entry for each page of the page table (16 pages)

16 page $\times$ 4B = 64 B can be stored in one page.

2-level is enough. Allocate first 4-bit of VPN as page directory index

$PDEAddr = PageDirBase + (PDIndex * sizeof(PDE));$



$PTEAddr = (PDE.PFN \ll SHIFT) + (PTIndex * sizeof(PTE));$

Single level paging: 16 pages

Two level paging: 3 pages (Page directory; 2 page of 2 page tables)

- Multi-level Page Table Control Flow

```
VPN = (VirtualAddress & VPN_MASK) >> SHIFT;
```

```
(Success, TlbEntry) = TLB_Lookup(VPN);
```

```
if (Success == True) {
```

```
    if (CanAccess(TlbEntry, ProtectBits) == True)
```

```
        Offset = VirtualAddress & OFFSET_MASK;
```

```
        PhysAddr = (TlbEntry.PFN << SHIFT) | Offset;
```

```
        Register = AccessMemory(PhysAddr);
```

```
    } else RaiseException(PROTECTION_FAULT);
```

```
} else { // perform the full multi-level lookup
```

```
    PDIndex = (VPN & PD_MASK) >> PD_SHIFT;
```

```
    PDEAddr = PDBR + (PDIndex * sizeof(PDE));
```

```
    PDE = AccessMemory(PDEAddr);
```

```
if (PDE.Valid == False)
```

```
    RaiseException(SEGMENTATION_FAULT);
```

```
else { // PDE is valid:
```

```
    PTIndex = (VPN & PT_MASK) >> PT_SHIFT;
```

```
    PTEAddr = (PDE.PFN << SHIFT) + (PTIndex * sizeof(PTE));
```

```
    PTE = AccessMemory(PTEAddr);
```

```
    if (PTE.Valid == False)
```

```
        RaiseException(SEGMENTATION_FAULT);
```

```
    else if (CanAccess(PTE.ProtectionBits) == False)
```

```
        RaiseException(PROTECTION_FAULT);
```

```
    else {
```

```
        TLB_Insert(VPN, PTE.PFN, PTE.ProtectionBits);
```

```
        Retry_Instruction();
```

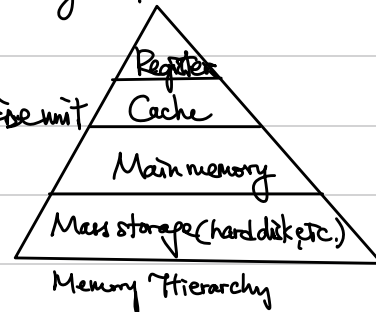
```
    }
```

• Inverted Page Tables

- Keeping a single page table that has an entry for each physical page of the system
- Entry indicates the process using the page, and the virtual page of that process mapped to the physical page

• Swapping Mechanisms

- OS stashes away portion of address space that are not in great demand
- Hard disk drive reserves a swap space for moving pages
- Swap space: for moving pages, OS remembers swap space in page size unit
- **Present Bit: Indicating that the page is in physical memory / disk**



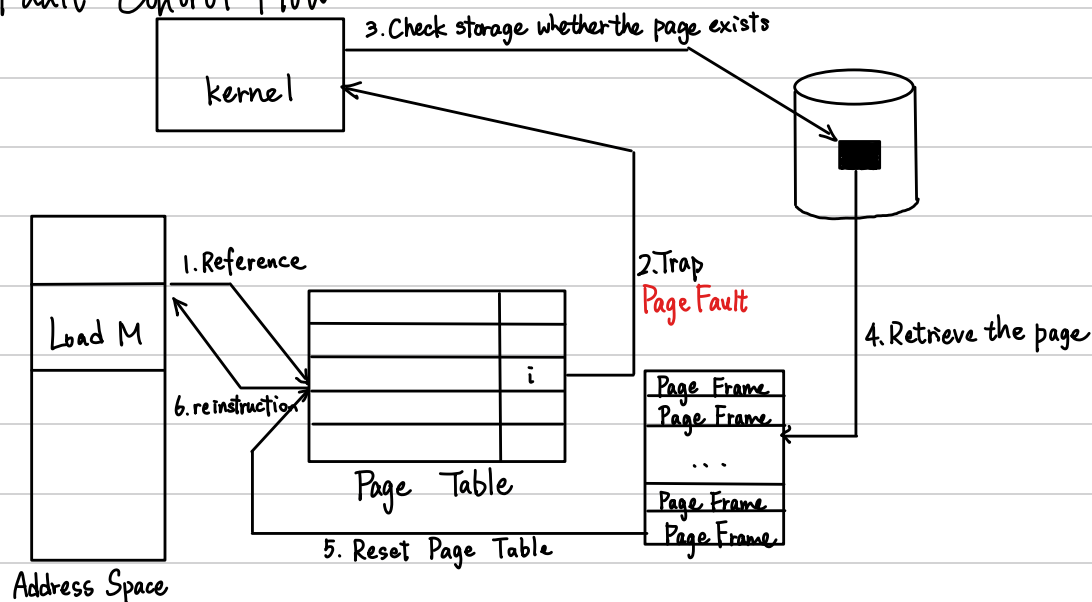
• Concepts

- Page fault: Accessing page that is not in physical memory
If a page is not present and has been swapped to disk, the OS needs to swap the page back into memory in order to handle the page fault
- Page replacement: The OS picks a page to evict/replace, known as page-replacement

• Page Replacement

- fewer than LW (low watermark) pages available, a background thread free memory runs
- thread evicts pages until HW (high watermark) pages are available

• Page Fault Control Flow



Software Control Flow

```

PFN = FindFreePhysicalPage();
if (PFN == -1)
    PFN = EvictPage();
DiskRead(PTE.DiskAddr, PFN);
PTE.Present = 1;
PTE.PFN = PFN;
RetryInstruction();
    
```

Find a physical frame for soon-to-be-faulted-in page
Use replacement algorithm

• Swapping Policies

- Minimize cache miss, improve average memory access time (AMAT)

$$AMAT = (P_{hit} \times T_m) + (P_{miss} \times T_d)$$

time for memory access time for disk access

- Optimal replacement policy: replace the page that will be accessed furthest-in-future (greedy)
↑
not known in real world

- FIFO: evict the first page on the head of the queue

Fails to consider the frequency & recency

Belady's Anomaly: hit rate decrease when cache is larger

Performance depends on randomness

- Random: picks a random page to evict

- Least-Frequently-Used (LFU)

Motivation: If a page has been accessed many times, it should not be replaced

- Least-Recently-Used (LRU)

Motivation: The more recently a page has been accessed, the more likely it will be accessed again

- Clock Algorithm

- Requires use bit: when a page is referenced, use bit is set to 1.

Hardware never clears the bits.

- All pages are arranged in a list, with a clock hand pointing to a page (check eviction)

When use_bit = 1, clear use_bit and advance pointer

Otherwise, evict the page.

- Dirty Pages

When a page is modified, simply set the dirty bit to 1 in the cache

When a dirty page is evicted, write back to disk

- Pre-fetching

OS guesses a page is about to be used and brings it in cache ahead of time

- Clustering, Grouping

Perform a single large write for a number of pending writes

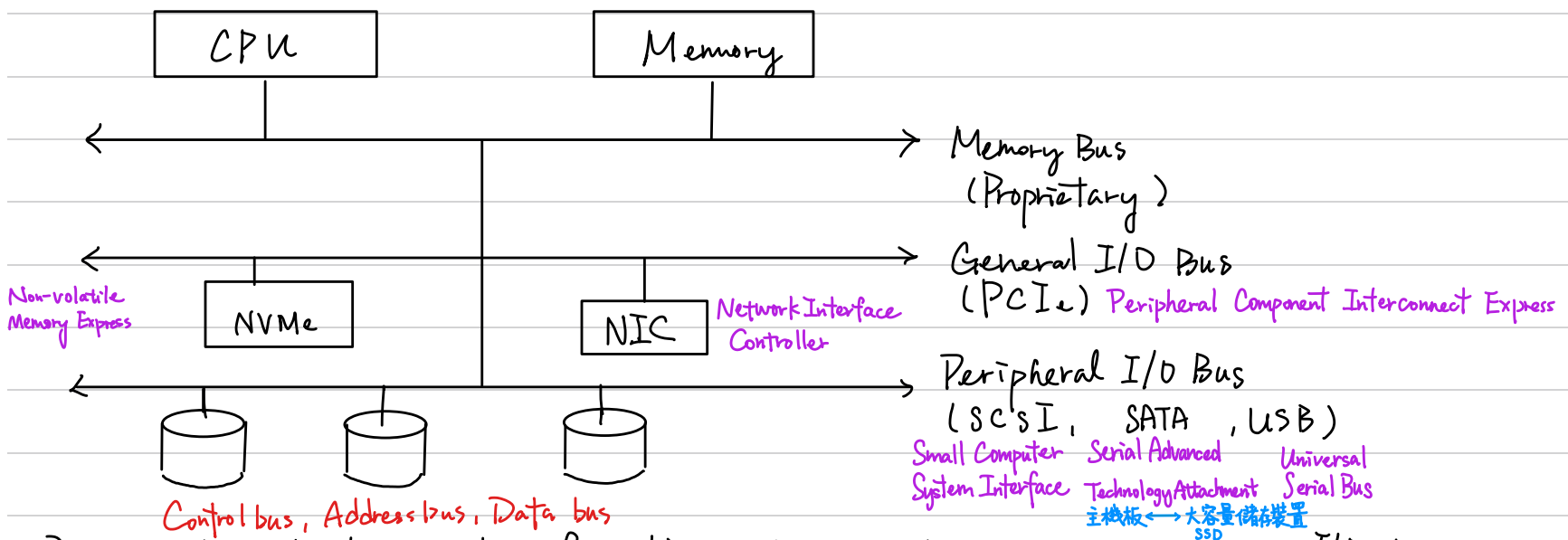
- Thrashing

Memory is oversubscribed: the memory demands of the set of running processes exceeds the available physical memory $\Rightarrow$ Only run a subset of processes

• I/O Devices

- Input / Output, interact with computer systems

• Prototypical System Architecture



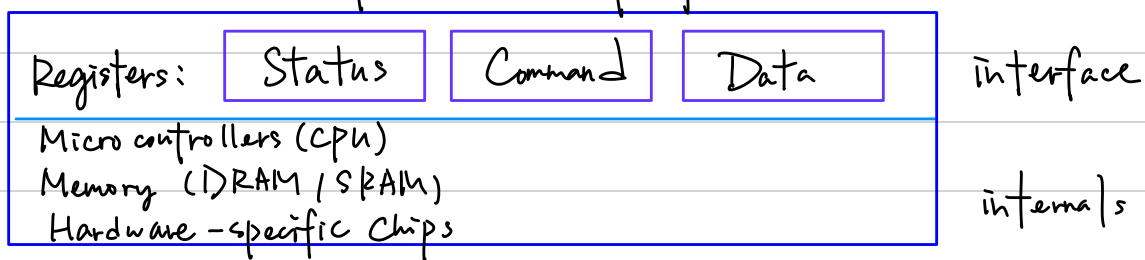
• Bus: datapaths to enable information exchange, between CPU, RAM and I/O devices

• I/O bus: datapath that connects a CPU to an I/O device

connected to I/O devices by hardware components: I/O ports, interfaces, device controllers

• Canonical Device

- Hardware interface allows the system software to control its operation
- Internals are implementation-specific



• Polling (Prefer fast devices)

Waits until the device is ready; responsive and simple, wastes a lot of CPU

• Interrupts (Favors slow devices)

Context-switched when waiting

I/O issues an interrupt to wake the sleeping process

- CPU is over-burdened due to memory copying from disk

Ask **DMA (Direct Memory Access)** to handle the memory copying request without wasting the CPU time. (paired with interrupts)

- OS - Device I/O Interaction

Communication [① I/O instructions: a way for the OS to send data to specific device registers
② Memory-mapped I/O:

OS directly loads/stores to the device instead of main memory

Interfaces - ③ Abstraction: Encapsulate specifics of device interaction

build a file system work on top of various devices

User mode: library calls $\rightarrow$ I/O handled by kernel

Kernel mode: Generic block interface [block read/write]

Specific block interface [protocol-specific read/write]

• Hard Disk Drive (HDD)

Performance Measurement

- Full rotation delay: time required per rotation = R

- Rotation delay: $\frac{R}{2}$ *Tracing a specific sector*

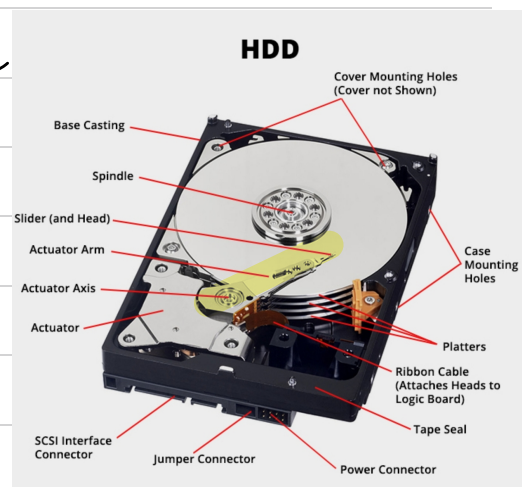
- Seek time: Acceleration $\rightarrow$ Coasting $\rightarrow$ Deceleration $\rightarrow$ Settling time

Placing the slider to the correct track

Track buffer cache: allows drive to quickly respond to requests

$\rightarrow$ write-back: add a dirty bit to cache, acknowledge the write is completed

write-through: directly apply changes to cache and disk



• I/O Time

$$T_{I/O} = T_{seek} + T_{rotation} + T_{transfer}$$

I/O Rate

$$R_{I/O} = \frac{\text{Size Transfer}}{T_{I/O}}$$

Example: 4KB Random Write

Cheetah 15K.5

RPM

15,000

Avg. Seek

4 ms

Max. Transfer

125 MB/s

$$f = 15000 \text{ rpm} = 250 \text{ rps}$$

$$T_{rotation} = \frac{1}{2f} = \frac{1}{2 \times 250} = 2 \text{ (ms)}$$

$$T_{seek} = 4 \text{ (ms)}$$

$$T_{transfer} = \frac{4}{125 \times 10^3} = 0.03 \text{ (ms)} = 30 \text{ (}\mu\text{s)}$$

$$T_{I/O} = T_{rotation} + T_{seek} + T_{transfer} = 2 + 4 + 0.03 \approx 6 \text{ (ms)}$$

$$R_{I/O} = \frac{4 \text{ KB}}{6 \text{ ms}} = 0.66 \text{ (MB/s)}$$

• Abstractions

- File: data with properties (name, type, permissions)

Described by "inode", index node, a data structure that describes a FS object

- Directory: A list of <user-readable filename, low-level name> pairs
(inode number)

• File System Interface

- Creating a File

open system call with O_CREAT flag

```
int fd = open("foo", O_CREAT | O_WRONLY | O_TRUNC, S_IRUSR | S_IWUSR);
```

O_CREAT - create a file

O_RDONLY/O_WRONLY - only read / write to that file while opened

O_TRUNC - make the file size zero (remove existing content)

O_LARGEFILE

A file descriptor fd is returned, which is then used to access files

A file descriptor table in PCB manages the files that are opened

• Tables for Files in Unix/Linux systems

- File description table

Per process: one entry for each fd opened by this process

Flags controlling the operations of a fd

- Open file table

Global, system-wide, one entry for each file opened

File offset, access mode

- Inode table

System-wide, one entry per inode in FS

File type, permissions, properties

- Intra-process Duplicated File Descriptors

Multiple fds referring to the same OFD — by dup() / dup2()

- Inter-process Duplicated File Descriptors

Two processes may have fds referring to the same OFD — by fork()

- Duplicated Opened File Descriptors

Two processes may have fds referring to distinct OFDs, or open file entries, that refer to the same inode

- Offset

The position of the file of where to start read and write.

Allocated when opened and adjusted after update

```
off_t lseek(int fd, off_t offset, int whence);
```

R/W to specific
offset in file

SEEK_SET: to offset bytes; SEEK_CUR: to its current location plus offset bytes; SEEK_END: to the size of the file plus offset bytes

- Persistency

`write()`: write data to buffer, later save it to storage

force writes immediately to disk by `fsync()`

Newly-created file should fully synchronize back to disk → call `fsync()` on the file and parent directory

• Simple File System

- File system

Disk is divided into blocks

① Data region: stores user data

② Inode table: size, owner

③ Allocation Structures: track if inodes / data blocks are free

Data bitmap / Inode bitmap

④ Super block: Number of inodes, begin location of inode table, etc.

Inode structure = indexed allocation

Small files does not require index blocks;

Large files indirection formed by index blocks that contains addresses of blocks that contain data

Directory structure

Record length: maximum length of the path that can be stored in a directory entry

String length: length of the path

• Caching and Buffering for File Systems

Cache popular blocks to reduce I/O

① Static partitioning: 10% of DRAM for block cache storing fd, 90% of page frame for VM

② Dynamic partitioning: page cache, merge virtual memory and buffer cache
physical page hosts either a page in process space or a file block

• Fast File System

- Disk-aware structures/policies

divide disks into cylinder groups / block groups → decreases seek time

- Data Structures for each cylinder group

① A copy of super block (reliability)

② Inode bitmap, data bitmap

③ Inodes, data block

- Placement of directories

Find a cylinder group with a low number of allocated directories and a high number of free inodes

- Placement of files

Same as directory